

CS-600: Software Design and Development

5-1 Short Paper: Mapping User Tasks to Application Architecture

Malgorzata Debska

Southern New Hampshire University

Matthew Scuteri

May 18, 2026

Customer Request Management System

Creating a software system requires understanding how its users interact with the application and how the software enables such interaction. Customer request management system in this case is meant to enable customer and administrative communication, request management, tracking of request updates and storing data securely. Automatic updating of data and storing logs of actions performed on the request should also be considered. As per the application requirements, customer data storage in database, request management capabilities, user and administrator communication capabilities, and secure coding should be implemented in the application. To create a structured application, it is important to identify CRUD operations and task-based actions first. CRUD operations are focused on data management, while task-based actions are aimed at performing particular tasks. In turn, functions are divided into several layers of the application that are associated with user interaction, business logic, and data storage, respectively. This structure ensures application maintainability and scalability.

CRUD Operations and User Tasks

CRUD means creating, reading, updating and cutting records in databases. Such functions are widely used in database-driven applications. Chandrasekaran and Krishnan (2022) note that CRUD functions are often used in data management applications that are directly connected to databases. There are several CRUD operations performed by customers and administrators using the customer request management system. Customers create new requests and updates in the system. Administrators perform reading of customer requests, watching of requests history, and

tracking of the request status. Both customers and administrators might perform updated actions about request statuses, or any other data related to the request. Administrators might drop requests, that are duplicates or false ones. Even though CRUD operations are essential to the functioning of applications, they are not thought of by users. Users always think of tasks they perform rather than database actions. For instance, customers might want to give a request, get some feedback, and communicate with administrators. Administrators will try to perform requests, resolve customer issues, record their actions, and inform customers about the request status. Tasks performed by users involve several CRUD operations at once. For instance, changing the request status into "resolved" means that there is a need to perform several actions: update the request record, record the time of update, record notes, and inform the customer. Task-based systems are more representative of user intentions than database-focused systems. According to Fowler, M. (2002), task-based design stands for user intention rather than database actions.

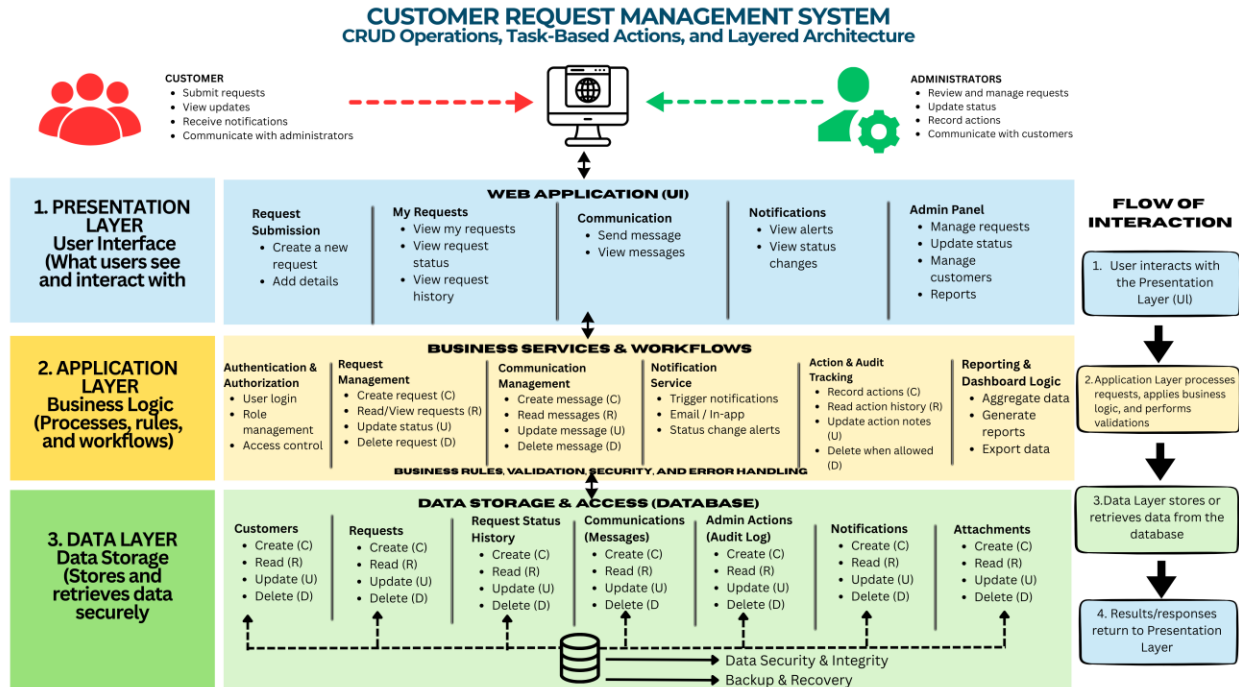
Alignment Between System Actions and User Goals

The system has been designed to serve both customers and administrators and to help them reach their goals. Customers want the system to be easy-to-use and reliable. They expect the system to provide quick access to request information and to be able to communicate with administrators. The system offers the above capabilities enabling customers to create requests, check request updates, and get notifications on the request status. Administrators use the system to organize request management and perform other relevant activities. Administrative duties include analyzing requests, changing their status, documenting all actions performed, and informing customers about the request status. Task-based design allows administrators to perform their tasks quickly and efficiently as the system combines all actions in particular workflows.

For example, instead of performing multiple actions related to a particular request (such as updating request records, adding time stamps and other information), the administrator just needs to click the button "Resolve Request." After that, all required CRUD operations are performed automatically by the system. Thus, task-based design increases the efficiency of administrative activities and prevents mistakes.

Layered Architecture Framework

Customer request management system has been designed using layered architecture. It includes presentation, application, and data layers. This approach improves the application's maintainability and allows developers to update components of the system without influencing the others. The presentation layer is the part of the system users interact with. The presentation layer includes forms, dashboards, request pages, communication screens, and other elements that provide users with the possibility to create requests, watch updates, and communicate with administrators. This layer presents data and receives input.



Application layer includes the business logic of the system. This layer handles processing requests, validating data entered by customers and administrators, applying security rules, managing workflows, and sending notifications to users. If the customer sends the request in the system, the application layer confirms information provided, saves it into the database, and sends confirmation notifications. Layered architecture is beneficial for organization and separation of system concerns per Chandrasekaran and Krishnan (2022). The data layer includes the database where all information users and their requests are stored. As per the application requirements, data should be stored in database using secure coding techniques.

Relationship Between CRUD Operations, User Tasks and Architecture

The relationship between CRUD operations, task-based actions and layered architecture is

important as each of these concepts helps to improve the system functionalities. CRUD operations offer technical capabilities for managing data in databases. Task-based actions are based on CRUD operations and represent workflows corresponding to user tasks. Layered architecture makes it possible to separate functions among layers to simplify the system. Adding new functionality to the system becomes easier as developers need to add new code to the application layer. Task-based design makes the system user-friendly as it is designed to meet users' needs and tasks. Moreover, layered architecture is beneficial for improving security. Validation, authentication, and request processing procedures are performed in the application layer, which decreases the risk of data leakage. Thus, layered architecture makes the system more scalable, organized, and reliable.

To conclude, a combination of CRUD operations and task-based actions in customer request management stem is needed to satisfy users' needs. CRUD operations provide the possibility of creating, reading, updating, and removing information in the database. Task-based actions stand for these operations in workflows that correspond to tasks and actions performed in the business. Layered architecture allows for separating application components into three layers (presentation, application, and data). This improves application maintainability, organization, scalability, and security. Customers get the opportunity to communicate with administrators and track request status, while administrators get the opportunity to perform their tasks more efficiently.

References

Chandrasekaran, P., & Krishnan, K. (2022). *Domain-driven design with Java: A practitioner's guide: Create simple, elegant, and valuable software solutions for complex business problems*. Packt Publishing.

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison Wesley Professional.